

Rapport de Projet — Maintenance Prédicative Industrielle

Système Intelligent Multi-Modèles pour la Maintenance Prédicative Industrielle

Projet M2 Data Science — EFREI 2025-26

Épreuve certifiante RNCP36739 — Bloc 4

Table des matières

1. Introduction et contexte
 2. Description du dataset
 3. Analyse Exploratoire des Données (EDA)
 4. Méthodologie
 5. Modèles implémentés
 6. Résultats et comparaison
 7. Analyse biais / variance et risques d'overfitting
 8. Interprétabilité du modèle final
 9. Dashboard décisionnel
 10. Analyse critique et limites
 11. Conclusion et recommandations
-

1. Introduction et contexte

1.1 Problématique industrielle

Dans les environnements industriels modernes, les équipements (CNC, pompes, compresseurs, bras robotiques) génèrent en permanence des données issues de capteurs physiques : vibrations, température moteur, pression, courant électrique, vitesse de rotation (RPM). Ces signaux contiennent des patterns annonciateurs de défaillances, souvent imperceptibles à l'œil humain.

La maintenance corrective (réparation après panne) engendre des coûts importants :

- Arrêts de production non planifiés

- Coûts de réparation d'urgence élevés
- Risques pour la sécurité des opérateurs

La **maintenance prédictive** vise à anticiper ces pannes avant qu'elles surviennent, en exploitant les données capteurs via des algorithmes d'apprentissage automatique. L'enjeu est de passer d'une posture réactive ("la machine est tombée en panne, on la répare") à une posture proactive ("la machine va tomber en panne dans 24h, on intervient maintenant").

1.2 Objectifs du projet

Ce projet a pour objectif de développer un **MVP (Minimum Viable Product)** de plateforme de maintenance prédictive comprenant :

1. Un pipeline complet de préparation des données (nettoyage, encodage, normalisation)
2. Une comparaison rigoureuse de 4 modèles ML/DL (Machine Learning classique + Deep Learning)
3. Un dashboard décisionnel interactif accessible à un profil non technique
4. Une analyse d'interprétabilité (Feature Importance + SHAP)

1.3 Tâche prédictive retenue

Prédiction binaire de panne dans les 24 heures

- Variable cible : `failure_within_24h` (0 = pas de panne, 1 = panne imminente)
- Justification : tâche la plus directement exploitable opérationnellement. Elle permet de planifier des interventions préventives avec un délai d'action suffisant.
- Le dataset contient d'autres variables cibles possibles (`failure_type` , `rul_hours`) mais nous avons choisi de construire une solution complète et rigoureuse autour d'une seule tâche, conformément aux exigences du projet.

2. Description du dataset

Source : Kaggle — Industrial Machine Predictive Maintenance Dataset

Fichier : `predictive_maintenance_v3.csv`

2.1 Caractéristiques générales

Attribut	Valeur
Nombre d'enregistrements	24 042
Nombre de variables	15
Période couverte	2024 (données simulées)
Fréquence d'échantillonnage	Toutes les ~3 minutes
Types de machines	CNC, Pump, Compressor, Robotic Arm

2.2 Variables du dataset

Variable	Type	Description	Statut
timestamp	datetime	Horodatage de la mesure	Supprimé (identifiant)
machine_id	int	Identifiant de la machine	Supprimé (identifiant)
machine_type	string	Type de machine	Feature catégorielle
vibration_rms	float	Vibration RMS	Feature numérique
temperature_motor	float	Température moteur (°C)	Feature numérique
current_phase_avg	float	Courant électrique moyen (A)	Feature numérique
pressure_level	float	Niveau de pression	Feature numérique
rpm	float	Vitesse de rotation	Feature numérique
operating_mode	string	Mode opératoire (idle/normal/peak)	Feature catégorielle
hours_since_maintenance	float	Heures depuis dernière maintenance	Feature numérique
ambient_temp	float	Température ambiante (°C)	Feature numérique
rul_hours	float	Durée de vie restante estimée	Supprimé (data leakage)
failure_within_24h	int	Variable cible (0/1)	Cible
failure_type	string	Type de panne	Supprimé (data leakage)
estimated_repair_cost	int	Coût estimé de réparation	Supprimé (data leakage)

2.3 Justification de la suppression des variables leakage

Le **data leakage** (fuite de données) désigne l'utilisation d'informations qui ne seraient pas disponibles au moment d'une prédiction réelle. C'est l'une des erreurs les plus fréquentes en Data Science : elle produit des scores artificiellement élevés qui ne se reproduisent pas en production.

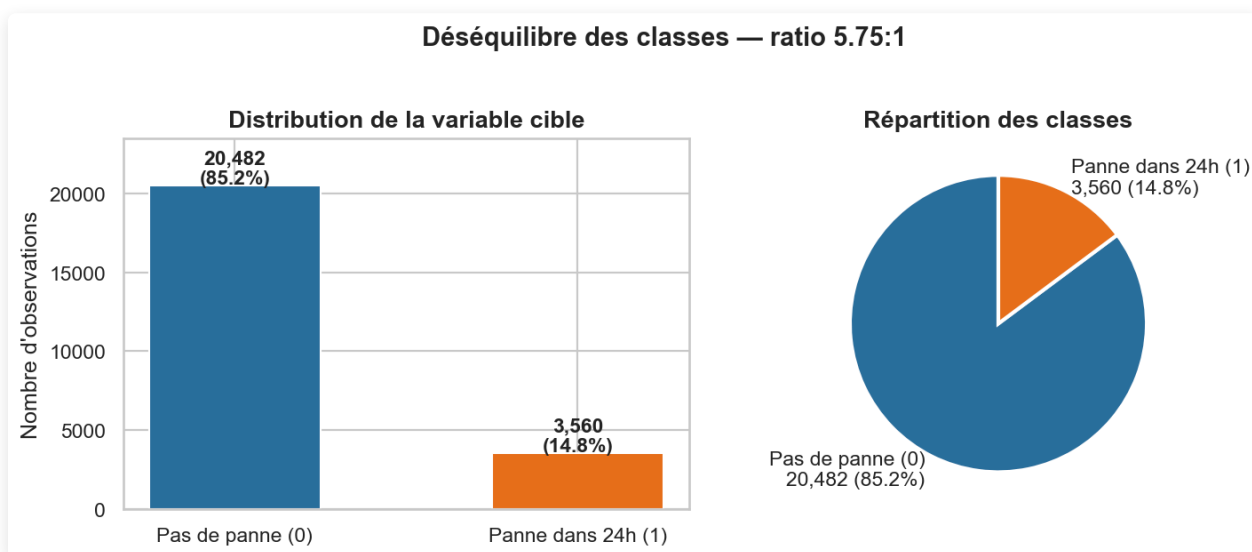
Trois variables ont été supprimées :

- **failure_type** : révèle directement qu'une panne est en cours → leakage évident. Si le modèle "voit" le type de panne, il sait déjà qu'il y a une panne.
- **ru1_hours** (durée de vie restante) : encode implicitement la cible. Si **ru1_hours < 24**, alors **failure_within_24h = 1** avec quasi-certitude. Corrélation mesurée : -0.25. Utiliser cette variable reviendrait à "tricher" en donnant au modèle la réponse déguisée.
- **estimated_repair_cost** : calculé à partir de la survenue d'une panne → non disponible avant la panne.

3. Analyse Exploratoire des Données (EDA)

3.1 Distribution de la variable cible

Classe	Nombre	Pourcentage
0 — Pas de panne	20 482	85.2%
1 — Panne dans 24h	3 560	14.8%



Le dataset présente un **déséquilibre significatif** (ratio ~5.75:1). Ce déséquilibre est réaliste dans un contexte industriel : les pannes sont des événements rares. Sans traitement, les modèles tendraient à prédire systématiquement "pas de panne" et afficheraient 85% d'accuracy sans jamais détecter une panne réelle — ce qui est parfaitement inutile.

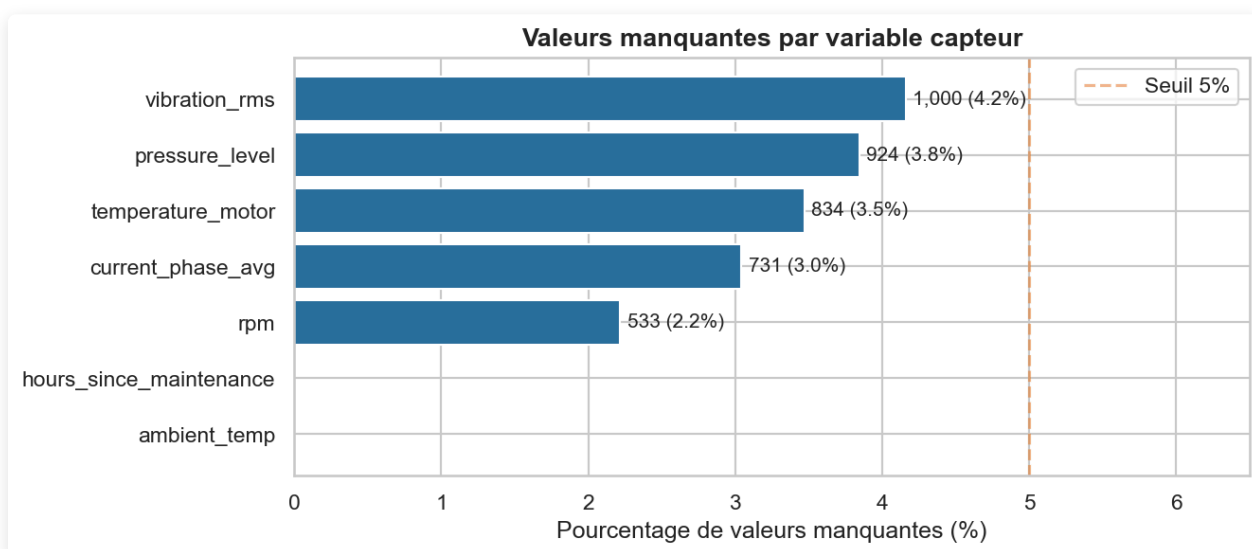
3.2 Répartition par type de machine

Machine	Observations
CNC	~25%
Pump	~25%
Compressor	~25%
Robotic Arm	~25%

La répartition est équilibrée entre les 4 types de machines. Cela garantit que les modèles ne seront pas biaisés vers un type de machine en particulier.

3.3 Valeurs manquantes

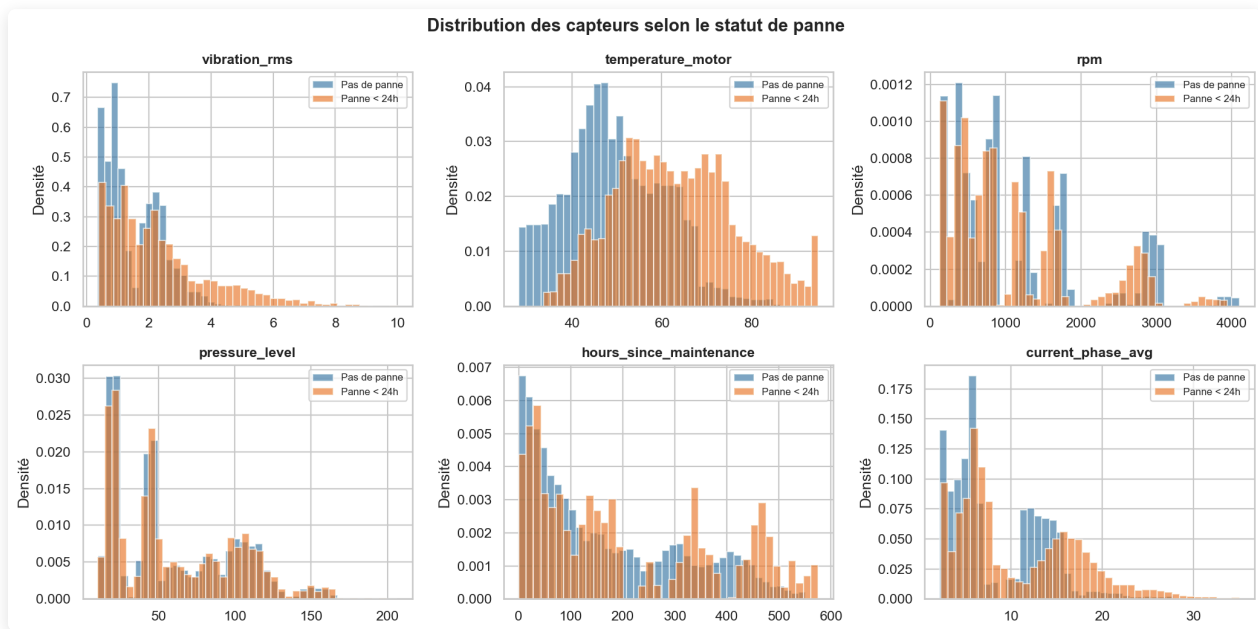
Variable	Valeurs manquantes	%
vibration_rms	1 000	4.2%
temperature_motor	834	3.5%
current_phase_avg	731	3.0%
pressure_level	924	3.8%
rpm	533	2.2%



Les valeurs manquantes représentent entre 2% et 4% de chaque capteur, probablement dues à des défaillances temporaires de capteurs ou des interruptions de communication. Une **imputation par la médiane** a été choisie car elle est insensible aux valeurs extrêmes

caractéristiques des données industrielles (pics de vibration, surchauffe), contrairement à la moyenne.

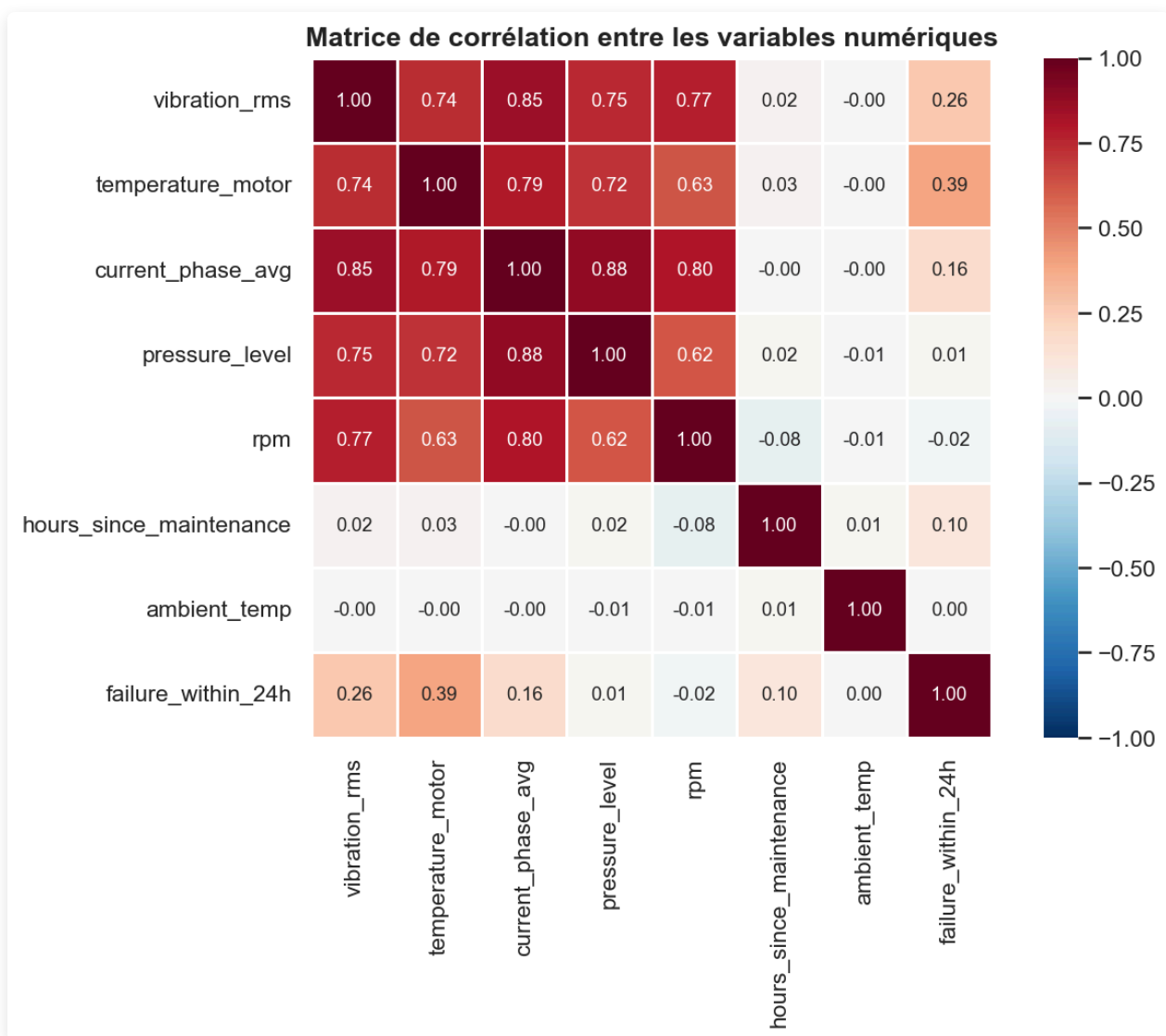
3.4 Distributions des capteurs par statut



L'analyse des distributions révèle des patterns distincts :

- **vibration_rms** : Les observations de classe 1 présentent des valeurs systématiquement plus élevées. Une vibration anormale est un précurseur physique classique de dégradation mécanique (roulements usés, balourd, desserrage).
- **temperature_motor** : Une surchauffe moteur est fortement associée aux pannes imminentes.
- **hours_since_maintenance** : Les machines dont la dernière maintenance est ancienne présentent un risque accru — cohérent avec l'usure progressive.
- **operating_mode** : Le mode **peak** est sur-représenté dans les observations de panne.

3.5 Matrice de corrélations



La matrice de corrélation ne révèle pas de multicolinéarité forte entre les features. Cela valide la pertinence de conserver toutes les variables dans les modèles. Les corrélations les plus notables avec la cible (`failure_within_24h`) concernent `vibration_rms` et `temperature_motor` .

4. Méthodologie

4.1 Pipeline de préparation des données

```
Données brutes (24 042 lignes, 15 colonnes)
↓
Suppression des colonnes (identifiants + data leakage)
↓
Séparation features / cible
↓
Split stratifié 80/20 (train: 19 233 | test: 4 809)
↓
ColumnTransformer (ajusté sur TRAIN uniquement, appliqué sur TEST)
├─ Numériques (7 variables) : SimpleImputer(median) → StandardScaler
└─ Catégorielles (2 variables) : SimpleImputer(mode) → OneHotEncoder
```

Le mot "**stratifié**" signifie que la proportion de pannes (14.8%) est préservée identiquement dans le train set et le test set. Sans stratification, par malchance, les pannes pourraient se retrouver davantage dans un ensemble que dans l'autre, faussant les résultats.

4.2 Prévention du data leakage — sklearn Pipelines

L'ensemble du preprocessing est intégré dans des **sklearn Pipelines**. Cela garantit que toutes les statistiques de preprocessing (médiane pour l'imputation, moyenne/écart-type pour le StandardScaler, catégories pour l'OneHotEncoder) sont calculées **uniquement sur les données d'entraînement**, puis appliquées (transformées) sur le test set sans que le modèle ait jamais "vu" les données de test pendant l'entraînement.

4.3 Stratégie de gestion du déséquilibre de classes

Modèle	Stratégie	Explication
Logistic Regression	<code>class_weight='balanced'</code>	sklearn recalcule des poids inversement proportionnels à la fréquence de chaque classe
Random Forest	<code>class_weight='balanced'</code>	Idem — chaque panne compte 5.75× plus qu'une non-panne
XGBoost	<code>scale_pos_weight=5.75</code>	Paramètre natif : ratio 20 482 / 3 560 ≈ 5.75
MLP	<code>early_stopping=True</code>	Arrête l'entraînement quand les performances se dégradent, évitant l'overfitting sur la classe majoritaire

4.4 Métriques d'évaluation — définitions et formules

Les 4 cas de figure possibles pour chaque prédiction :

	Prédit : PAS de panne	Prédit : PANNE
Réel : PAS de panne	✓ TN — Vrai Négatif	✗ FP — Faux Positif (fausse alerte)
Réel : PANNE	✗ FN — Faux Négatif (panne ratée !)	✓ TP — Vrai Positif

Dans un contexte industriel, le FN est le pire cas : le modèle dit "tout va bien" alors qu'une panne arrive → arrêt brutal, coûts d'urgence, risques de sécurité.

Recall (métrique prioritaire) = $TP / (TP + FN)$ — Parmi toutes les vraies pannes, combien le modèle en détecte ?

Precision = $TP / (TP + FP)$ — Parmi toutes les alertes, quelle proportion est justifiée ?

F1-Score = $2 \times (Precision \times Recall) / (Precision + Recall)$ — Moyenne harmonique, critère de sélection du modèle

ROC-AUC — Aire sous la courbe ROC. La courbe représente, pour tous les seuils possibles, le Recall (axe Y) vs le taux de faux positifs (axe X). AUC = 1 → modèle parfait. AUC = 0.5 → modèle aléatoire.

4.5 Validation croisée

Une **cross-validation stratifiée 5-fold** a été appliquée sur le modèle retenu (XGBoost). Le dataset est divisé en 5 parties : le modèle est entraîné 5 fois, chaque fois testé sur une partie différente. Un écart-type faible entre les 5 scores confirme que le modèle généralise réellement et n'est pas surpris sur un découpage particulier.

5. Modèles implémentés

5.1 Logistic Regression — Modèle baseline

Principe interne : calcule une combinaison linéaire pondérée des features ($z = w_1 \times \text{vibration} + w_2 \times \text{température} + \dots$), puis applique la fonction sigmoïde $\sigma(z) = 1/(1+e^{(-z)})$ pour obtenir une probabilité entre 0 et 1. Le modèle apprend les coefficients w pendant l'entraînement.

Paramètres : `C=1.0`, `solver='lbfgs'`, `max_iter=1000`, `class_weight='balanced'`

Forces : très interprétable (coefficient positif = la feature augmente le risque), rapide, robuste

Limites : modèle linéaire → ne capture pas "si vibration ET température sont simultanément élevées" (interactions non linéaires)

Rôle : baseline de référence. Si ce modèle était le meilleur, le problème serait linéairement séparable et les modèles complexes seraient inutiles.

5.2 Random Forest — Modèle ensembliste (Bagging)

Principe interne : construit 200 arbres de décision **indépendants**, chacun entraîné sur un sous-échantillon aléatoire des données (bagging). Chaque arbre pose des questions binaires (`vibration_rms > 2.3 ?`) jusqu'à une décision finale. La prédiction finale = **moyenne des 200 probabilités** (vote).

La double randomisation (sous-ensemble d'observations + sous-ensemble de features à chaque nœud) fait que les arbres font des erreurs différentes — en moyennant, les erreurs s'annulent.

Paramètres : `n_estimators=200` , `max_depth=15` , `class_weight='balanced'`

Forces : capture les non-linéarités, robuste aux outliers, feature importance native

Limites : modèle volumineux (21 MB), moins performant que le boosting sur ce dataset

5.3 XGBoost — Gradient Boosting

Principe interne : construit les arbres **séquentiellement** — chaque arbre corrige les erreurs du précédent. L'algorithme utilise le **gradient de la fonction de perte** pour savoir où concentrer l'effort de correction. Avec 200 arbres et un `learning_rate=0.1` , chaque arbre corrige 10% des erreurs restantes de manière contrôlée.

Différence clé avec Random Forest : les arbres ne sont **pas indépendants**, ils se corrigent mutuellement.

Paramètres : `n_estimators=200` , `max_depth=6` , `learning_rate=0.1` , `scale_pos_weight=5.75`

Forces : meilleure performance sur données tabulaires, régularisation L1/L2 intégrée, déploiement léger (~600 KB)

Limites : hyperparamétrage plus complexe

5.4 MLP — Deep Learning (Réseau de neurones multicouche)

Principe interne : réseau de neurones artificiels avec 3 couches cachées. Chaque neurone calcule $z = \sum(w_i \times x_i) + b$ puis applique ReLU : $f(z) = \max(0, z)$. L'entraînement utilise la

rétropropagation (backpropagation) — à chaque exemple, l'erreur remonte couche par couche pour ajuster tous les poids.

Architecture : Input (11 features) → 128 neurones → 64 → 32 → Output (sigmoïde → probabilité)

Paramètres : `hidden_layer_sizes=(128, 64, 32)` , `activation='relu'` , `alpha=1e-4` , `early_stopping=True`

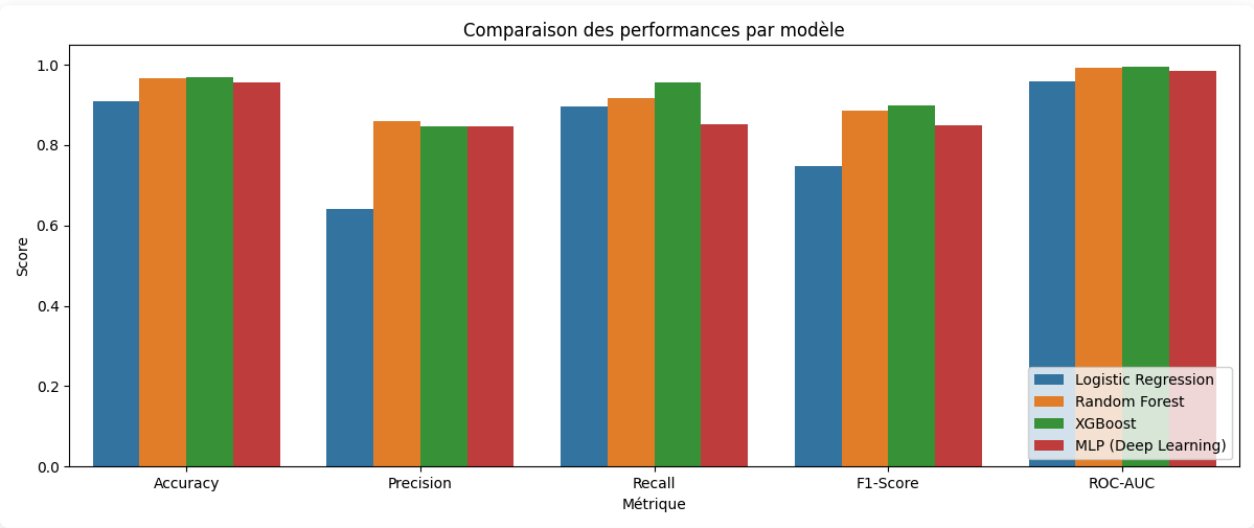
Forces : capture des interactions complexes sans feature engineering manuel

Limites : boîte noire, sensible au manque de données, sensible à l'initialisation aléatoire

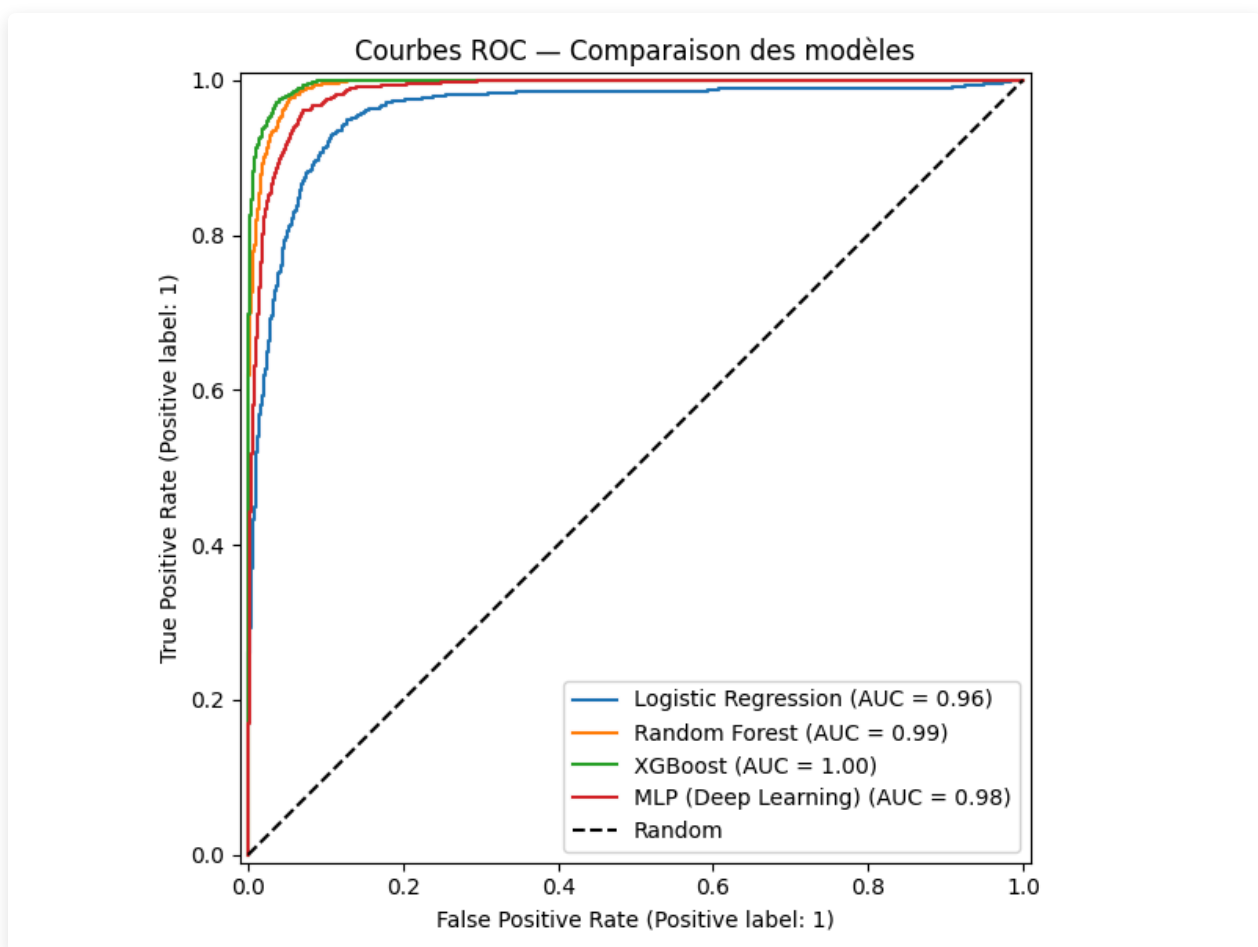
6. Résultats et comparaison

6.1 Tableau des performances (ensemble de test, n=4 809)

Modèle	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.910	0.641	0.895	0.747	0.959
Random Forest	0.965	0.859	0.916	0.887	0.993
XGBoost ✓	0.968	0.847	0.955	0.898	0.996
MLP (Deep Learning)	0.955	0.847	0.853	0.850	0.984



6.2 Courbes ROC — comparaison des 4 modèles



6.3 Analyse des résultats

Logistic Regression :

La Precision faible (0.641) indique de nombreuses fausses alertes. La limitation linéaire du modèle ne permet pas de capturer les interactions complexes entre capteurs. **Son rôle est confirmé : tous les autres modèles le surpassent significativement** (+0.14 F1 pour Random Forest).

Random Forest :

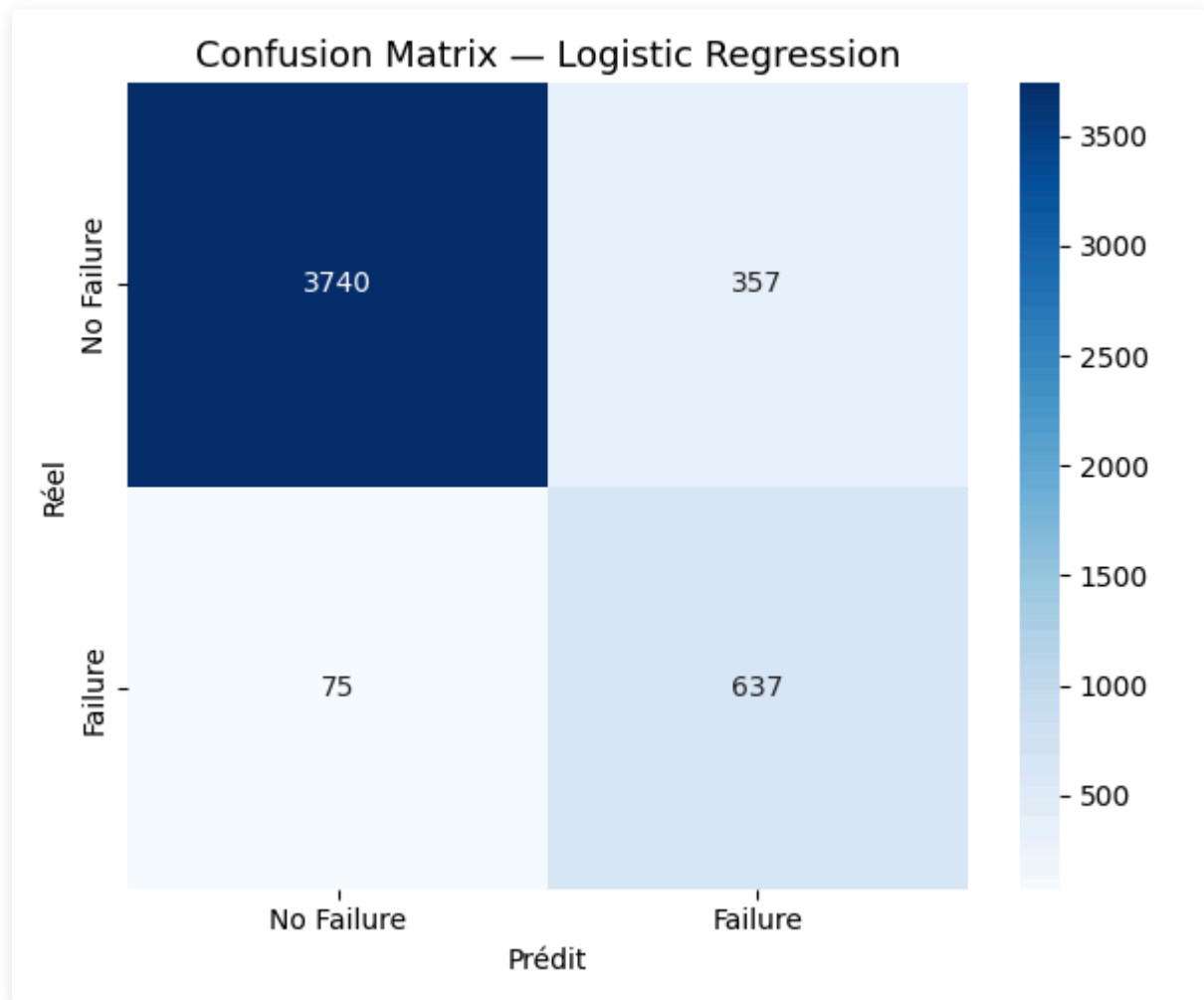
Excellentes performances (F1=0.887, ROC-AUC=0.993). Le gain de +0.14 F1 par rapport à la régression logistique confirme la présence de relations non linéaires dans les données. Cependant, XGBoost le surpasse sur tous les critères, et son volume (21 MB vs 600 KB) pénalise le déploiement.

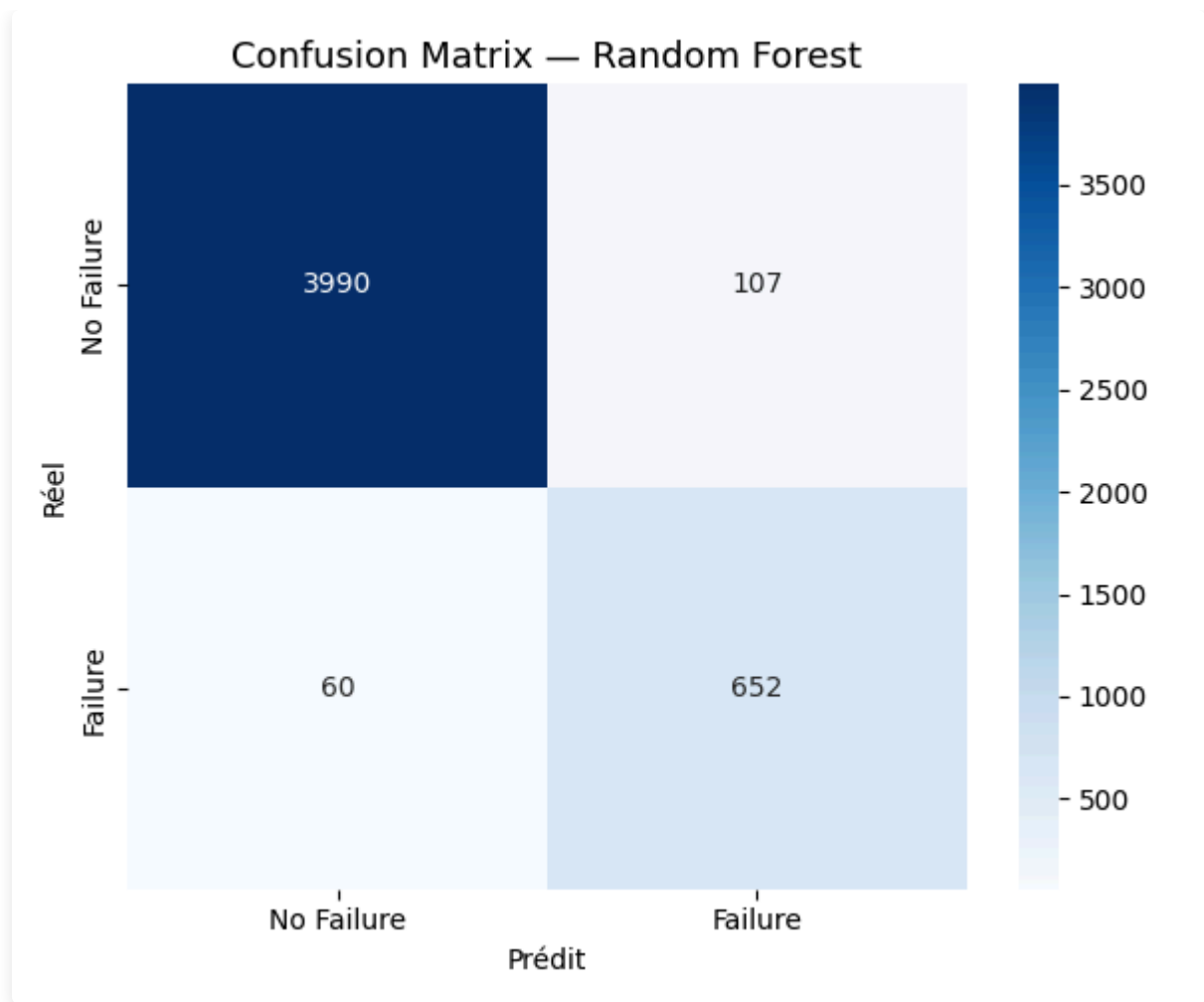
XGBoost (modèle retenu) :

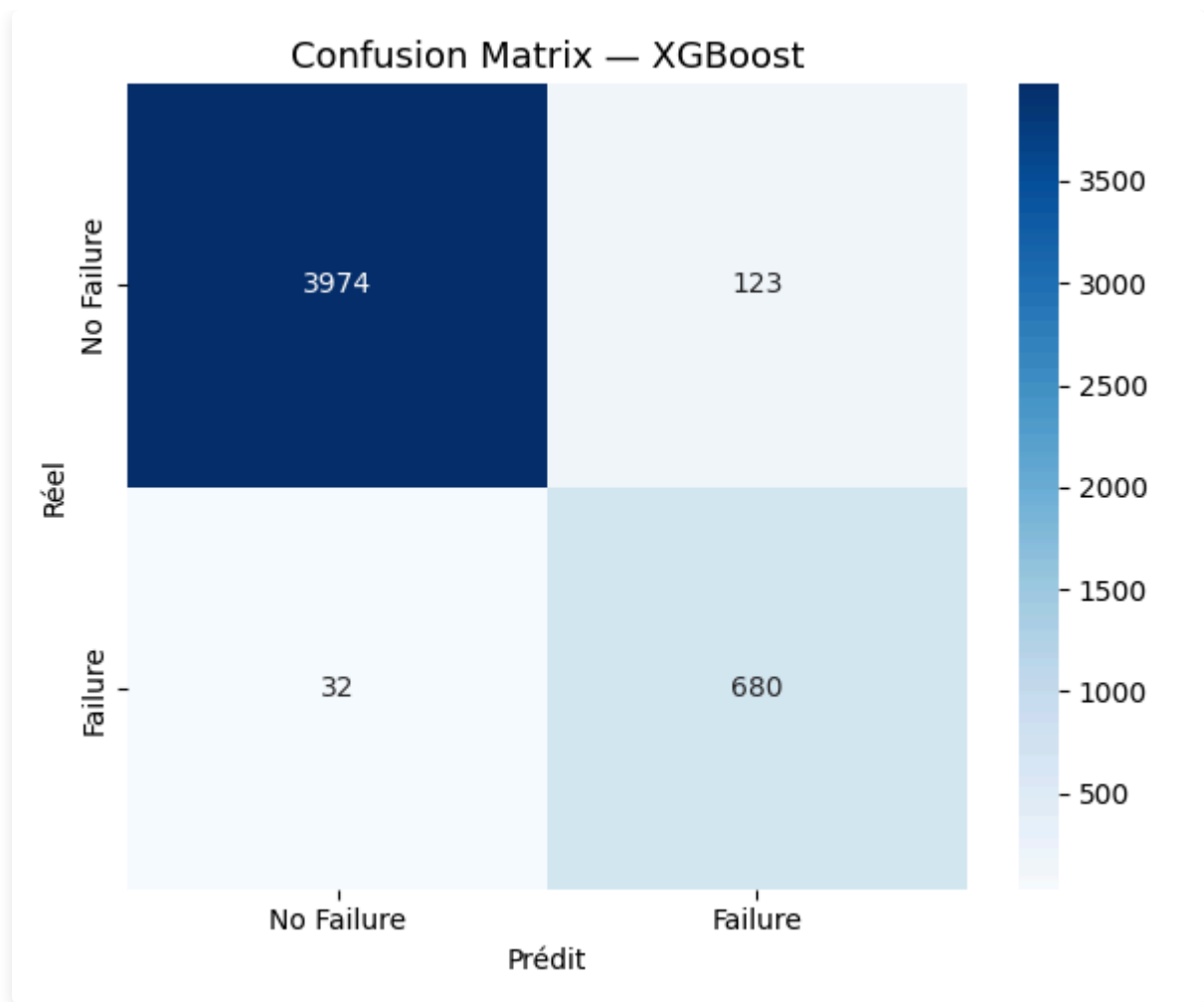
Meilleur compromis toutes métriques confondues. Le **Recall de 0.955** signifie que 95.5% des pannes réelles sont détectées — soit 19 pannes sur 20. Le ROC-AUC de 0.996 indique une quasi-parfaite capacité à distinguer les deux classes.

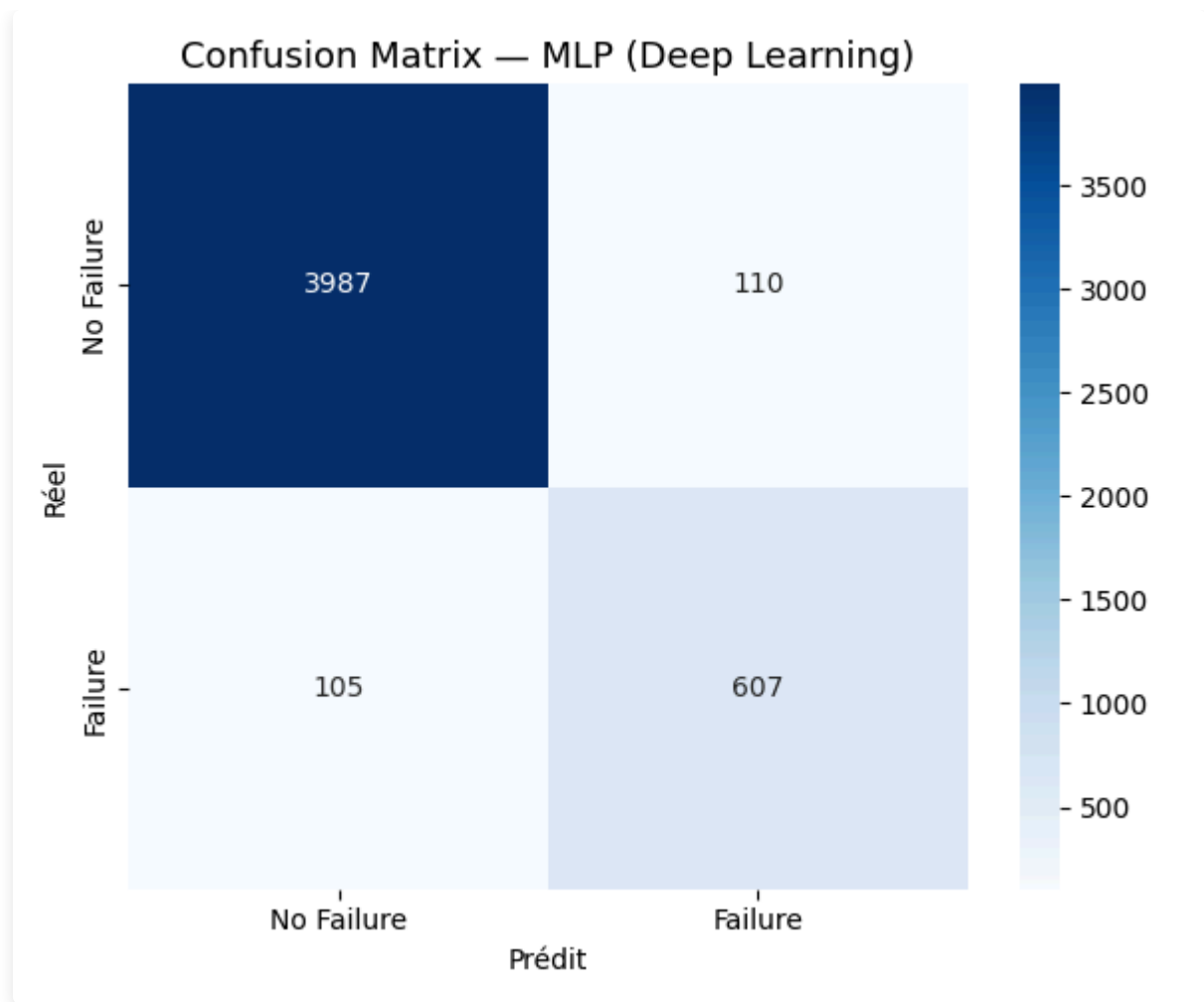
MLP (Deep Learning) :

F1=0.850, inférieur à XGBoost. Avec 24 000 observations et 9 features structurées, les arbres boostés ont l'avantage. Le Deep Learning excelle sur des données non structurées (images, texte) ou des séries temporelles longues.

6.4 Matrices de confusion — comparaison des 4 modèles







6.5 Cross-validation XGBoost (5-fold stratifié)

Fold	F1-Score
1	0.8930
2	0.8915
3	0.9105
4	0.9008
5	0.9169
Moyenne	0.9026
Écart-type	0.0099

L'écart-type de 0.01 confirme que le modèle est **stable** : les performances sont reproductibles sur n'importe quelle partition du dataset.

6.6 Justification du choix du modèle final

Critère	Évaluation
Performance (F1, ROC-AUC)	✓ Meilleur de tous les modèles
Recall (détection des pannes)	✓ 0.955 — crucial en contexte industriel
Stabilité (CV 5-fold)	✓ Faible variance entre folds (0.0099)
Gestion du déséquilibre	✓ <code>scale_pos_weight</code> natif
Interprétabilité	✓ Feature importance + SHAP (TreeExplainer)
Coût computationnel	✓ Entraînement ~30 secondes
Déploiement	✓ Sérialisation joblib légère (~600 KB vs 21 MB pour RF)

7. Analyse biais / variance et risques d'overfitting

7.1 Le compromis biais / variance

Tout modèle de Machine Learning est confronté à un compromis fondamental :

- **Biais élevé (underfitting)** : le modèle est trop simple, ne capture pas les patterns → erreurs élevées sur train ET test
- **Variance élevée (overfitting)** : le modèle mémorise le bruit du train set → très bon sur train, mauvais sur test
- **Bon équilibre** : performances proches et élevées sur train et test

7.2 Analyse par modèle

Modèle	Biais	Variance	Diagnostic
Logistic Regression	Élevé	Faible	Underfitting — modèle trop simple pour les non-linéarités
Random Forest	Faible	Faible	Bon équilibre — le bagging réduit la variance naturellement
XGBoost	Faible	Très faible	Excellent équilibre — régularisation L1/L2 intégrée
MLP (Deep Learning)	Modéré	Modéré	Risque d'overfitting sur classe majoritaire → <code>early_stopping</code> appliqué

7.3 Mesures prises pour contrôler l'overfitting

- **Random Forest** : `max_depth=15` limite la profondeur des arbres ; `class_weight='balanced'` équilibre l'apprentissage
- **XGBoost** : `max_depth=6` (arbres peu profonds), `learning_rate=0.1` (correction progressive), régularisation L1/L2 native
- **MLP** : `early_stopping=True` arrête l'entraînement dès que la validation loss cesse de diminuer ; `alpha=1e-4` applique une régularisation L2 sur les poids

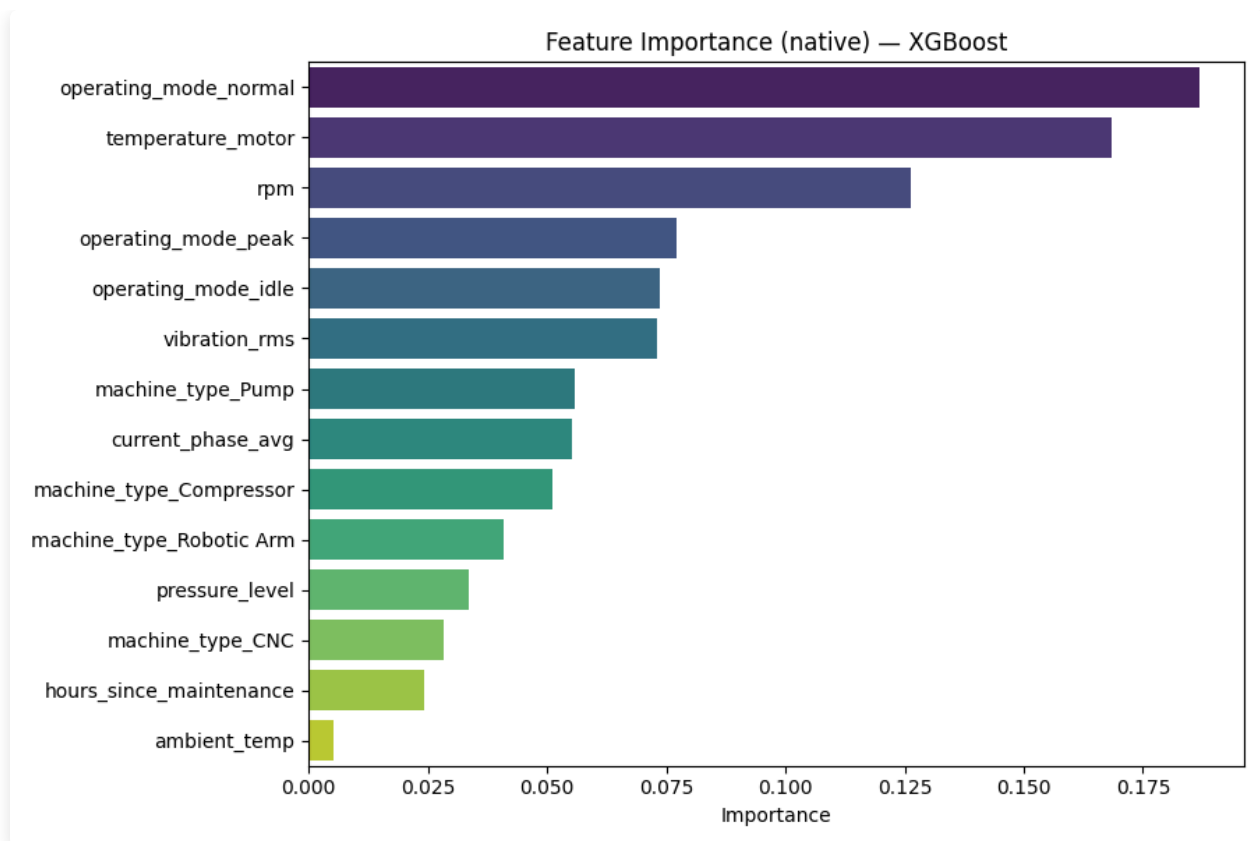
7.4 Preuve empirique de l'absence d'overfitting

La cross-validation (5-fold) sur XGBoost donne $F1 = 0.9026 \pm 0.0099$ sur données jamais vues pendant l'entraînement. Le score sur le test set est 0.898 — très proche. Un modèle en overfitting montrerait un écart important entre train et test. La faible variance (0.0099) confirme la bonne généralisation.

8. Interprétabilité du modèle final

8.1 Feature Importance native (XGBoost)

L'importance des variables est calculée par le **gain moyen** apporté par chaque variable lors des splits dans tous les arbres. Plus une variable est utilisée fréquemment et apporte un gain élevé, plus elle est "importante".



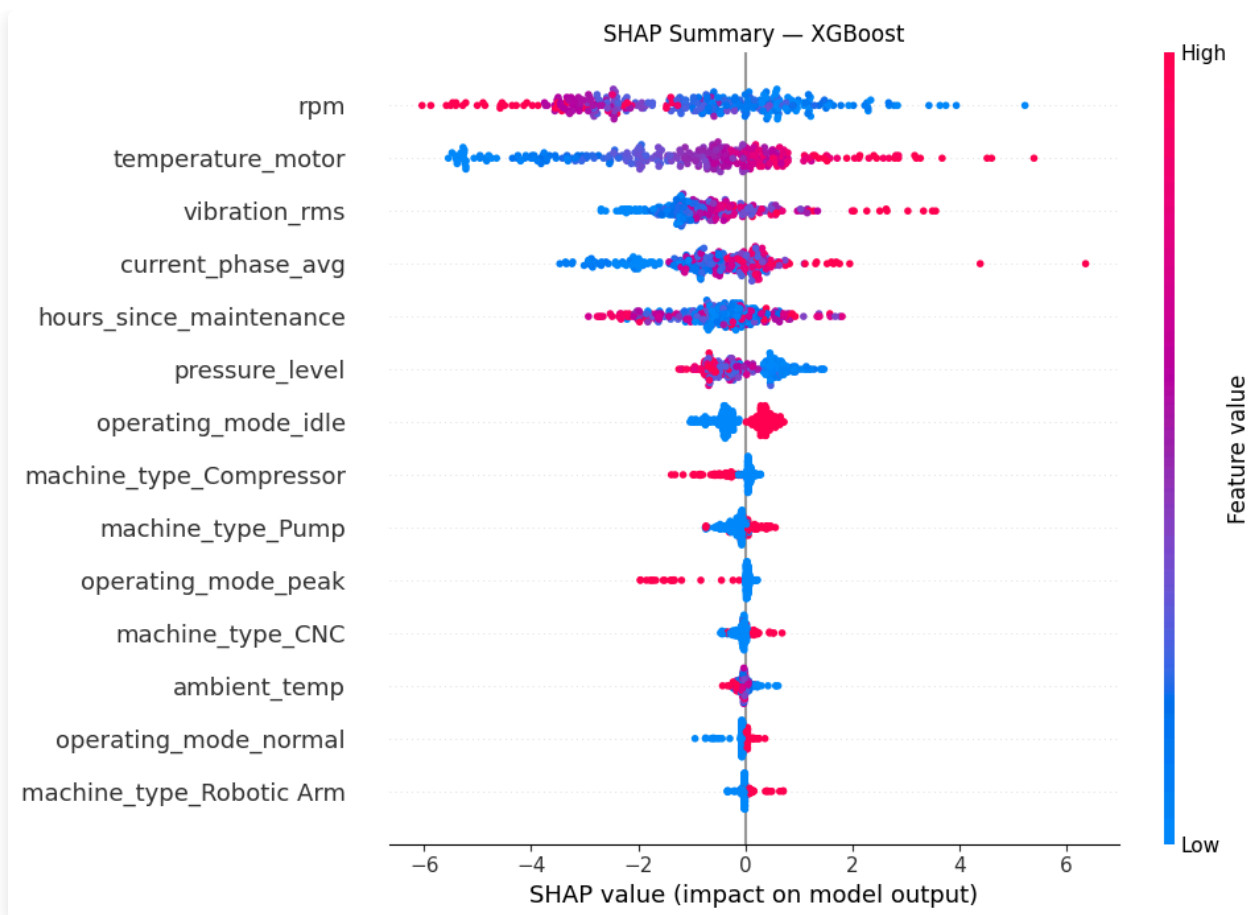
Top 5 variables :

1. **vibration_rms** — indicateur principal de dégradation mécanique (roulements, balourd, desserrage)
2. **temperature_motor** — précurseur de défaillances thermiques (isolation endommagée, surcharge)
3. **hours_since_maintenance** — mesure de l'usure accumulée depuis la dernière intervention
4. **rpm** — stress mécanique lié à la vitesse de rotation
5. **pressure_level** — anomalie hydraulique ou pneumatique

Ces résultats sont **cohérents avec la physique des machines industrielles**, ce qui valide que le modèle apprend des patterns physiquement sensés — et non du bruit statistique.

8.2 SHAP (SHapley Additive exPlanations)

La Feature Importance donne une vision globale. SHAP va plus loin en expliquant **chaque prédiction individuelle**, en décomposant la prédiction en contributions de chaque feature (basé sur la théorie des jeux coopératifs).



Lecture du graphique :

- **Axe X positif** → contribue à prédire "panne"
- **Axe X négatif** → contribue à prédire "pas de panne"
- **Rouge** = valeur élevée de la feature | **Bleu** = valeur faible
- **Ordre vertical** = importance globale (feature la plus impactante en haut)

Différence Feature Importance vs SHAP :

Feature Importance	SHAP
Vision globale (macro)	Vision locale (par observation)
"Quelle feature est la plus utilisée ?"	"Pourquoi cette machine est-elle à risque ?"
Basée sur la fréquence d'utilisation	Basée sur la contribution marginale (théoriquement fondé)

Utilité opérationnelle : un responsable maintenance peut répondre à "Pourquoi cette machine est-elle classée à haut risque ?" → La vibration RMS est anormalement élevée → vérifier les roulements.

9. Dashboard décisionnel

Le dashboard Streamlit a été conçu comme un **outil décisionnel autonome**, distinct des visualisations d'analyse scientifique (EDA). Il cible un profil **responsable maintenance** non technique.

9.1 Architecture du dashboard

Onglet	Contenu	Valeur métier
 EDA	Distributions, corrélations, manquants	Compréhension des données machine
 Modèles	Tableau, ROC, matrices de confusion	Confiance dans le système
 Prédiction	Formulaire capteurs → score de risque	Usage opérationnel quotidien
 Interprétabilité	Feature importance, SHAP	Explicabilité des alertes

9.2 Cas d'usage — Prédiction en temps réel

Un responsable maintenance entre les valeurs des capteurs d'une machine suspecte. Le dashboard affiche :

- **Score de risque coloré** : ● ÉLEVÉ (probabilité > 60%)
- **Probabilité exacte** : ex. 78.3%
- **Recommandation** : Intervention recommandée dans les 24h

9.3 Lancement

```
streamlit run dashboard/app.py → http://localhost:8501
```

10. Analyse critique et limites

10.1 Limites du dataset

- **Données simulées** : les performances pourraient être inférieures sur données industrielles réelles (bruit, capteurs défaillants, variabilité inter-machines)
- **Absence de features temporelles** : chaque enregistrement est traité indépendamment. Une approche LSTM/séries temporelles pourrait capturer les tendances d'évolution des capteurs avant la panne

- **Pas de contexte machine** : âge, historique complet de pannes, qualité des pièces de maintenance non inclus

10.2 Limites méthodologiques

- **Seuil de décision fixe à 0.5** : en production, ce seuil devrait être ajusté selon le ratio coût panne / coût intervention. Un coût de panne 10× supérieur justifie de baisser le seuil à 0.3 pour maximiser le Recall
- **Pas de monitoring de dérive (data drift)** : en production, les distributions des capteurs évoluent (vieillessement), ce qui dégraderait progressivement les performances sans détection
- **Hyperparamètres non optimisés par GridSearch** : une optimisation par RandomizedSearchCV pourrait améliorer légèrement les résultats

10.3 Comparaison ML vs Deep Learning

Facteur	Impact sur MLP vs XGBoost
Taille du dataset (24k obs.)	MLP nécessite typiquement 100k+ observations pour exploiter sa capacité
Features tabulaires structurées	Les arbres de décision y sont naturellement adaptés
Features déjà bien définies	Peu de valeur ajoutée des représentations latentes apprises par le réseau
Données déséquilibrées	XGBoost gère mieux nativement via <code>scale_pos_weight</code>

Conclusion : XGBoost (F1=0.898) > MLP (F1=0.850). Résultat cohérent avec la littérature (Grinsztajn et al., 2022 : "Why tree-based models still outperform deep learning on tabular data").

11. Conclusion et recommandations

11.1 Bilan

Ce projet a démontré la faisabilité d'un système de maintenance prédictive basé sur le Machine Learning :

- **XGBoost** est le modèle retenu : F1 = **0.898**, Recall = **0.955** (19 pannes sur 20 détectées)
- Cross-validation confirme la stabilité : F1 = **0.9026 ± 0.0099**
- Pipeline sklearn anti-leakage (ColumnTransformer + Pipelines)

- Dashboard Streamlit opérationnel (4 onglets, prédiction temps réel)

11.2 Recommandations pour une mise en production

Recommandation	Priorité	Impact attendu
Ajuster le seuil de décision selon le ratio coût panne / coût intervention	Haute	+5–10% Recall
Intégrer des features temporelles (rolling mean 1h, 6h, 24h)	Haute	+3–5% F1 estimé
Monitoring de dérive des distributions (data drift)	Moyenne	Fiabilité long terme
Réentraînement périodique avec nouvelles données	Moyenne	Maintien des performances
API REST FastAPI pour intégration SCADA/ERP	Optionnel	Industrialisation

11.3 Impact métier estimé

Avec un Recall de 95.5%, le système permettrait :

- De **détecter 19 pannes sur 20** avant leur survenue
- De planifier des interventions préventives à moindre coût
- De réduire les arrêts non planifiés et leurs impacts sur la production

Rapport rédigé dans le cadre du Projet M2 Data Science — EFREI 2025-26

Validation RNCP36739 — Bloc 4 : Implémenter des méthodes d'intelligence artificielle